

SG90 Micro Servo

Hardware Overview

What is it?

The **SG90 Micro Servo** is a small, lightweight motor used to create precise movements. Unlike a standard DC motor that spins freely, a servo can be told exactly where to point or how fast to spin. They are used for everything from robotic arms to driving wheels.

Electrical Explanation

Servos are controlled by sending a specific electrical pulse to them, known as a **PWM signal** (similar to how we dim LEDs).

- Wires: The servo usually has three wires: **Brown (GND/0V)**, **Red (Power/5V)**, and **Orange (PWM Signal)**.
- Signal: The microcontroller sends pulses through the orange wire. The duration (width) of this pulse tells the servo what angle to hold or what speed to rotate at.

Key Hardware Facts

- **Connection**: Wired to a Digital PWM Pin (e.g. D6).
- **Servo IDs**: Supports up to 4 servos (ID 0 to 3) simultaneously.
- **Voltage**: 5V
- **Signal Flow**: Processor (Digital PWM Pin) → Orange Signal Wire → Internal Servo Controller → PWM Logic → DC Motor & Gearbox → Physical Rotation.

Two Types of Servos

The library supports two distinct behaviors:

1. Standard Positional (180°): Moves to a specific angle between 0° and 180° and holds that position. Used for joints, latches, or steering.
2. Continuous Rotation (360°): Spins continuously like a wheel. You control the speed and direction, not the position.

Software & Programming

kcomp_servoSG90.h

This library provides a simplified functional interface for controlling both positional and continuous rotation servos.

Usage: `#include <kcomp_servoSG90.h>`

Function Reference

Function Name	Description & Parameters
<code>bool initServo(uint8_t id, int pin, uint8_t type)</code>	Setup: Attaches a servo to a specific pin. ▪ <code>id</code>: The unique ID (0-3). ▪ <code>pin</code>: Digital PWM pin number (e.g., 6).

	▪ type: Use <code>SERVO_TYPE_180</code> or <code>SERVO_TYPE_360</code>.
void setServoAngle(uint8_t id, int angle)	Control (180° Only): Sets the target position. ▪ angle: Target in degrees (0 to 180).
void setServoSpeed(uint8_t id, int speed)	Control (360° Only): Sets direction and speed. ▪ speed: Percentage from -100 (backward) to 100 (forward). ▪ 0 = Stop
void stopServo(uint8_t id)	Stop: Halts the motor immediately. ▪ For 360°: Sets speed to 0. ▪ For 180°: No effect (motor holds current angle).

⚠ Pitfall: "Wrong Function, Wrong Servo"

The library includes safety checks. If you try to set an angle on a 360° servo, or set a speed on a 180° servo, the command will be ignored.

Why? Sending an "angle" command to a wheel servo would just make it spin at a specific weird speed, which is confusing. The library prevents this mistake.

Sample Program

Below is a simple program to test a standard 180° servo on ID 0.

```
#include <kcomp_servoSG90.h>

#define MY_SERVO_PIN 6
#define MY_SERVO_ID 0

void setup() {
    Serial.begin(115200);

    // 2. Initialize servo 0 on Pin 9 as a 180-degree type
    initServo(MY_SERVO_ID, MY_SERVO_PIN, SERVO_TYPE_180);
    Serial.println("Servo Ready!");
}

void loop() {
    // Move to 0 degrees
    setServoAngle(MY_SERVO_ID, 0);
    delay(1000); // Wait for the arm to get there

    // Move to 90 degrees (Middle)
    setServoAngle(MY_SERVO_ID, 90);
    delay(1000);

    // Move to 180 degrees
    setServoAngle(MY_SERVO_ID, 180);
    delay(1000);
}
```

 **Try it out:** You can find the full sample program **SimpleServoSG90** in the K-Comp examples folder!